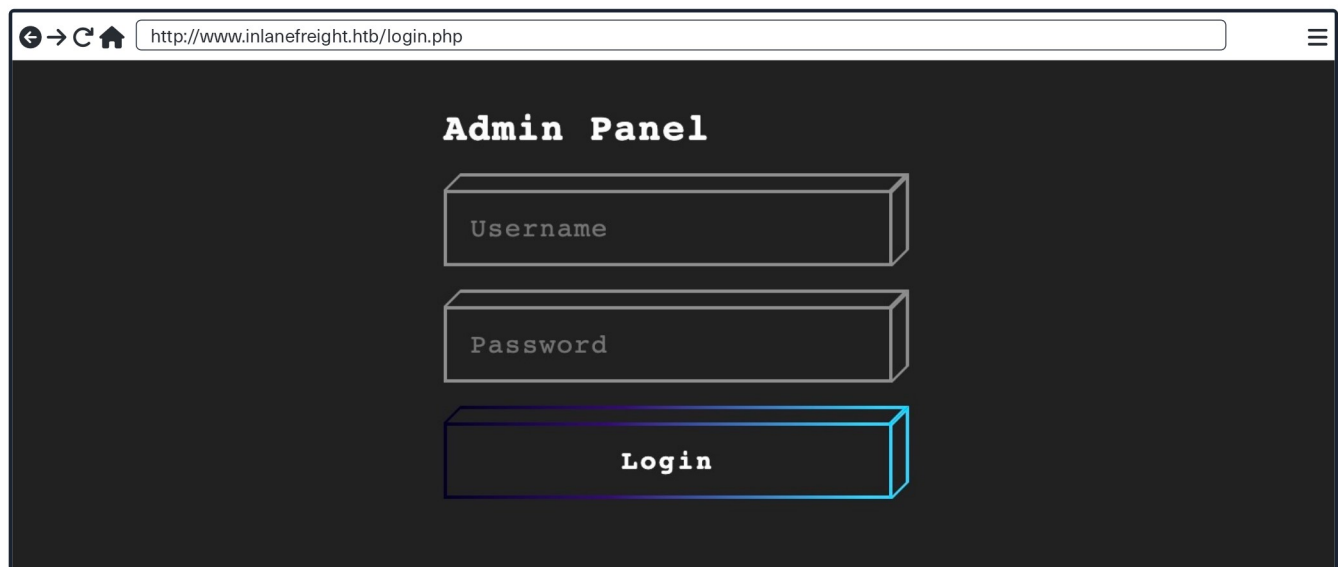


Hydra Modules

Since we found a login form on the webserver for administrators during our penetration testing engagement, it is a very interesting component to which we should try to gain access without generating much network traffic. Finally, with the admin panels, we can manage servers, their services, and configurations. Many admin panels have also implemented features or elements such as the [b374k shell](#) that might allow us to execute OS commands directly.

Login.php



The screenshot shows a web browser window with the address bar displaying `http://www.inlanefreight.htb/login.php`. The page content is titled "Admin Panel" and features three input fields: "Username", "Password", and a "Login" button. The "Login" button is highlighted with a red border.

To cause as little network traffic as possible, it is recommended to try the top 10 most popular administrators' credentials, such as `admin:admin`.

If none of these credentials grant us access, we could next resort to another widespread attack method called password spraying. This attack method is based on reusing already found, guessed, or decrypted passwords across multiple accounts. Since we have been redirected to this admin panel, the same user may have access here.

Brute Forcing Forms

Hydra provides many different types of requests we can use to brute force different services. If we use `hydra -h`, we should be able to list supported services:

Hydra Modules		
<pre>dbcypth0n@htb[/htb]\$ hydra -h grep "Supported services" tr ":" "\n" tr " " "\n" column -e</pre>		
Supported	ldap3[-{cram digest}md5][s]	rsh
services	memcached	rtsp
	mongodb	s7-300
adam6500	mssql	sip
asterisk	mysql	smb
cisco	nntp	smtp[s]
cisco-enable	oracle-listener	smtp-enum
cvs	oracle-sid	snmp
firebird	pcanywhere	socks5
ftp[s]	pcnfs	ssh
http[s]-{-head get post}	pop3[s]	sshkey

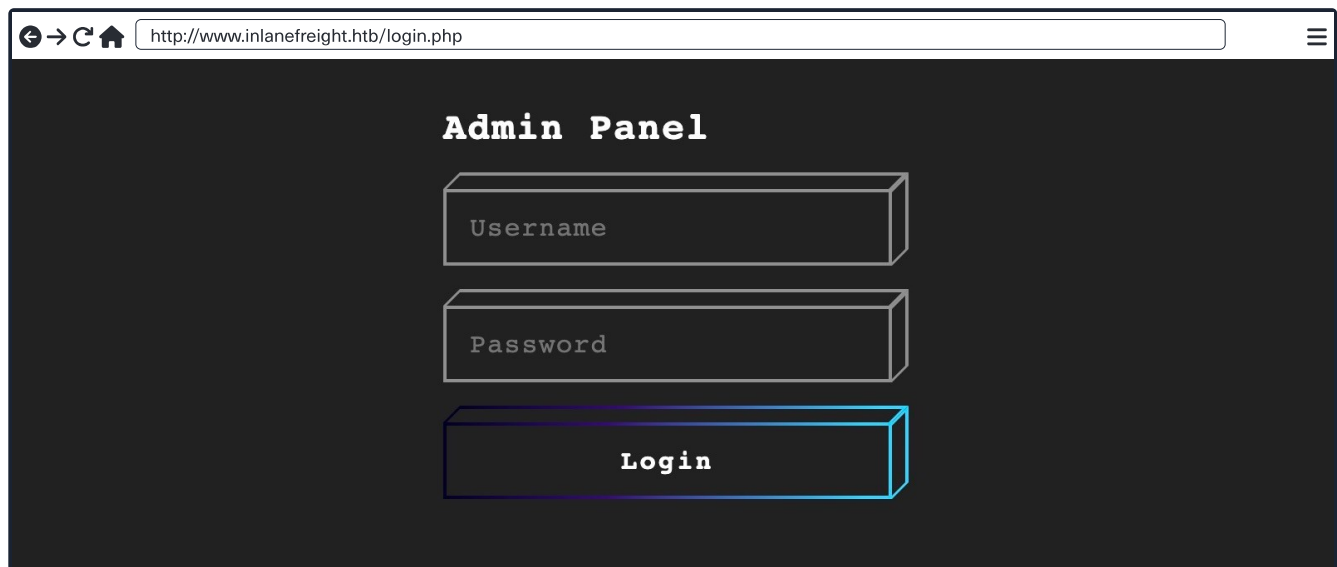
http[s]-{get post}-form	postgres	svn
http-proxy	radmin2	teamspeak
http-proxy-urlenum	rdp	telnet[s]
icq	redis	vmauthd
imap[s]	rexec	vnc
irc	rlogin	xmpp
ldap2[s]	rpcap	

In this situation there are only two types of **http** modules interesting for us:

1. **http[s]-{head|get|post}**
2. **http[s]-post-form**

The 1st module serves for basic HTTP authentication, while the 2nd module is used for login forms, like **.php** or **.aspx** and others.

Since the file extension is **".php"** we should try the **http[s]-post-form** module. To decide which module we need, we have to determine whether the web application uses **GET** or a **POST** form. We can test it by trying to log in and pay attention to the URL. If we recognize that any of our input was pasted into the **URL**, the web application uses a **GET** form. Otherwise, it uses a **POST** form.



When we try to log in with any credentials and don't see any of our input in the URL, and the URL does not change, we know that the web application uses a **POST** form.

Based on the URL scheme at the beginning, we can determine whether this is an **HTTP** or **HTTPS** post-form. If our target URL shows **http**, in this case, we should use the **http-post-form** module.

To find out how to use the **http-post-form** module, we can use the **"-U"** flag to list the parameters it requires and examples of usage:

Hydra Modules

```
dbcyph0n@htb[/htb]$ hydra http-post-form -U
```

<...SNIP...>

Syntax: <url>:<form parameters>:<condition string>[:<optional>[:<optional>]

First is the page on the server to GET or POST to (URL).

Second is the POST/GET variables ...SNIP... usernames and passwords being replaced in the **"^USER^"** and **"^PASS^"** placeholders

The third is the string that it checks for an ***invalid*** login (by default)

Invalid condition login check can be preceded by **"F="**, successful condition login check must be preceded by **"S="**.

<...SNIP...>

Examples:

```
"^USER^" " ^PASS^" " ^INVALID^" " ^S^" " ^F^"
```

```
"/login.php:user=^USER^&pass=^PASS^:incorrect"
```

In summary, we need to provide three parameters, separated by :, as follows:

- 1. URL path, which holds the login form
- 2. POST parameters for username/password
- 3. A failed/success login string, which lets hydra recognize whether the login attempt was successful or not

For the first parameter, we know the URL path is:

Hydra Modules
/login.php

The second parameter is the POST parameters for username/passwords:

Hydra Modules
/login.php:[user parameter]=^USER^&[password parameter]=^PASS^

The third parameter is a failed/successful login attempt string. We cannot log in, so we do not know how the page would look like after a successful login, so we cannot specify a success string to look for.

Hydra Modules
/login.php:[user parameter]=^USER^&[password parameter]=^PASS^:[FAIL/SUCCESS]=[success/failed string]

Fail/Success String

To make it possible for hydra to distinguish between successfully submitted credentials and failed attempts, we have to specify a unique string from the source code of the page we're using to log in. Hydra will examine the HTML code of the response page it gets after each attempt, looking for the string we provided.

We can specify two different types of analysis that act as a Boolean value.

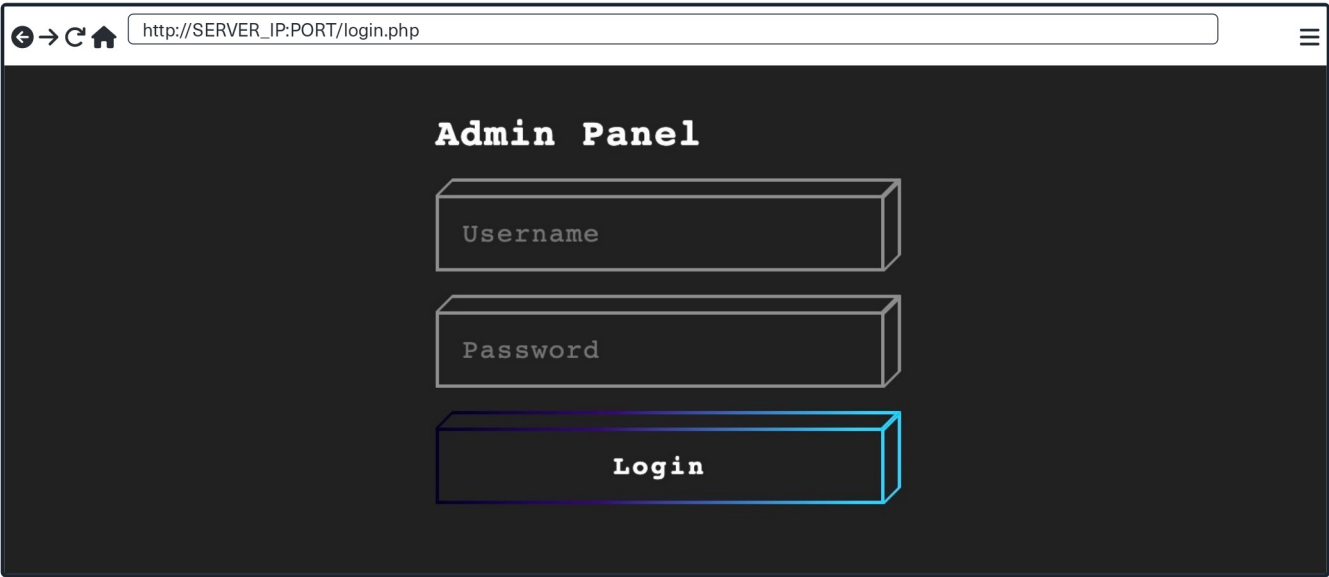
Type	Boolean Value	Flag
Fail	FALSE	F=html_content
Success	TRUE	S=html_content

If we provide a fail string, it will keep looking until the string is not found in the response. Another way is if we provide a success string, it will keep looking until the string is found in the response.

Since we cannot log in to see what response we would get if we hit a success, we can only provide a string that appears on the logged-out page to distinguish between logged-in and logged-out pages.

So, let's look for a unique string so that if it is missing from the response, we must have hit a successful login. This is usually set to the error message we get upon a failed login, like Invalid Login Details. However, in this case, it is a little bit trickier, as we do not get such an error message. So is it still possible to brute force this login form?

We can take a look at our login page and try to find a string that only shows in the login page, and not afterwards. For example, one distinct string is `Admin Panel`:



So, we may be able to use `Admin Panel` as our fail string. However, this may lead to false-positives because if the `Admin Panel` also exists in the page after logging in, it will not work, as `hydra` will not know that it was a successful login attempt.

A better strategy is to pick something from the HTML source of the login page. What we have to pick should be very *unlikely* to be present after logging in, like the **login button** or the *password field*. Let's pick the login button, as it is fairly safe to assume that there will be no login button after logging in, while it is possible to find something like `please change your password` after logging in.

We can click `[Ctrl + U]` in Firefox to show the HTML page source, and search for `login`:

Code: `html`

```
<form name='login' autocomplete='off' class='form' action='' method='post'>
```

We see it in a couple of places as title/header, and we find our button in the HTML form shown above. We do not have to provide the entire string, so we will use `<form name='login'`, which should be distinct enough and will probably not exist after a successful login.

So, our syntax for the `http-post-form` should be as follows:

Code: `bash`

```
"/login.php:[user parameter]=^USER^[password parameter]=^PASS^:F=<form name='login'"
```

← Previous

Next →

+10 Streak pts

✔ Mark Complete & Next

Cheat Sheet

Table of Contents

Introduction

Introduction to Brute Forcing

✔

Basic HTTP Auth Brute Forcing

Password Attacks



 Default Passwords



 Username Brute Force



Web Forms Brute Forcing

[Hydra Modules](#)

Determine Login Parameters

 Login Form Attacks

Service Authentication Attacks

Personalized Wordlists

 Service Authentication Brute Forcing

Skills Assessment

 Skills Assessment - Website

 Skills Assessment - Service Login

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left